

you have n books on the shelf, you'll have to check all n books. In this example, the runtime increases at the same rate as the number of books.

Some programmers also use *big Omega notation*, which describes the best-case scenario for an algorithm. For example, a $\Omega(n)$ algorithm performs at linear efficiency at its best. In the worst case, it might perform slower. Some algorithms encounter especially lucky cases where no work has to be done, such as finding driving directions to a destination when you're already at the destination.

Big Theta notation describes algorithms that have the same best- and worst-case order. For example, $\Theta(n)$ describes an algorithm that has linear efficiency at best *and* at worst, which is to say, it's an $O(n)$ and $\Omega(n)$ algorithm. These notations aren't used in software engineering as often as big O, but you should still be aware of their existence.

It isn't uncommon for people to talk about the "big O of the average case" when they mean big Theta, or "big O of the best case" when they mean big Omega. This is an oxymoron; big O specifically refers to an algorithm's worst-case runtime. But even though their wording is technically incorrect, you can understand their meaning irregardless.